



南開大學
Nankai University

数据结构实验报告七

堆和霍夫曼编码

学号：2412266

姓名：杨滢

专业：信息安全

学院：密码与网络空间安全学院

目录

1 题目一	2
1.1 题目表述	2
1.2 代码 1 的测试用例	2
1.3 思路	3
1.3.1 构造函数 myHeap(int n, T big, T small)	3
1.3.2 上浮操作 up(int pos)	3
1.3.3 下沉操作 down(int pos)	3
1.4 代码	4
1.5 测试用例截图	7
1.6 与本节实现的比较	8
1.6.1 本节书中的最大堆实现方式	8
1.6.2 本题的最大堆实现方式	8
2 题目二	8
2.1 题目表述	8
2.2 代码 2 的测试用例	8
2.3 思路	9
2.3.1 哈夫曼树构建	9
2.3.2 生成编码表	9
2.3.3 压缩处理	9
2.3.4 解压缩处理	9
2.4 代码	10
2.5 测试用例截图	15

1 题目一

1.1 题目表述

根据下面的假设, 重新编写类 `maxHeap` 的方法。

1. 在创建堆时, 创建者应该提供两个元素 `maxElement` 和 `miinElement`。堆中没有元素比
2. `maxElement` 大, 也没有元素比 `minElement` 小。
3. 一个 `n` 元素的堆需要一个数组 `heap[0:2n+1]`。
4. `n` 个元素按本节所描述的方法存储在 `heap[1:n]` 中。
5. `maxElement` 存储在 `heap[0]` 中。
6. `minElement` 存储在 `heap[n+1:2n+1:2n+1]` 中。

这些假设应该使 `push` 和 `pop` 的代码简化。通过实验将本练习的实现与本节的实现做比较。

1.2 代码 1 的测试用例

1. `empty()` -> true
2. `size()` -> 0
3. `push(50)` -> ok, size:1, top:50
4. `push(30)` -> ok, size:2, top:50
5. `push(999999999)` -> ok, size:3, top:999999999
6. `push(-999999999)` -> ok, size:4, top:999999999
7. `push(50)` -> ok, size:5, top:999999999
8. `push(10)` -> ok, size:6, top:999999999
9. `push(60)` -> full, size:6, top:999999999
10. `top()` -> ok, value:999999999
11. `empty()` -> false
12. `pop()` -> ok, value:999999999, size:5, top:50
13. `pop()` -> ok, value:50, size:4, top:50
14. `pop()` -> ok, value:50, size:3, top:30
15. `pop()` -> ok, value:30, size:2, top:10
16. `pop()` -> ok, value:10, size:1, top:-999999999
17. `pop()` -> empty, size:0

1.3 思路

通过在数组的特定位置预设极大值和极小值，让比较操作自动处理边界情况。

1.3.1 构造函数 myHeap(int n, T big, T small)

```
1 myHeap(int n, T big, T small) : maxSize(n), maxVal(big), minVal(small), cnt(0) {
2     arr.resize(2 * n + 2);
3     arr[0] = maxVal; // 设置左边界
4     for (int i = n + 1; i <= 2 * n + 1; i++) {
5         arr[i] = minVal; // 设置右边界
6     }
7 }
```

分配 $2n+2$ 的空间来容纳边界值和实际数据；初始化边界位置的值。

1.3.2 上浮操作 up(int pos)

```
1 void up(int pos) {
2     while (pos > 1 && arr[pos] > arr[pos / 2]) {
3         swap(arr[pos], arr[pos / 2]);
4         pos /= 2;
5     }
6 }
```

当 $pos=1$ （根节点）时， $arr[1] > arr[0]$ 比较的是实际元素与 $maxVal$ 。由于 $maxVal$ 是极大值，这个比较永远为假，循环自动终止。

1.3.3 下沉操作 down(int pos)

```
1 void down(int pos) {
2     while (pos * 2 <= cnt) {
3         int kid = pos * 2;
4         if (kid < cnt && arr[kid] < arr[kid + 1]) {
5             kid++; // 选择较大的孩子
6         }
7         if (arr[pos] >= arr[kid]) break;
8         swap(arr[pos], arr[kid]);
9         pos = kid;
10    }
11 }
```

当访问 $heap[n+1]$ 及之后的元素时，都是 $minElement$ 。与 $minElement$ 比较时，当前节点值肯定更大，自动停止下沉。

1.4 代码

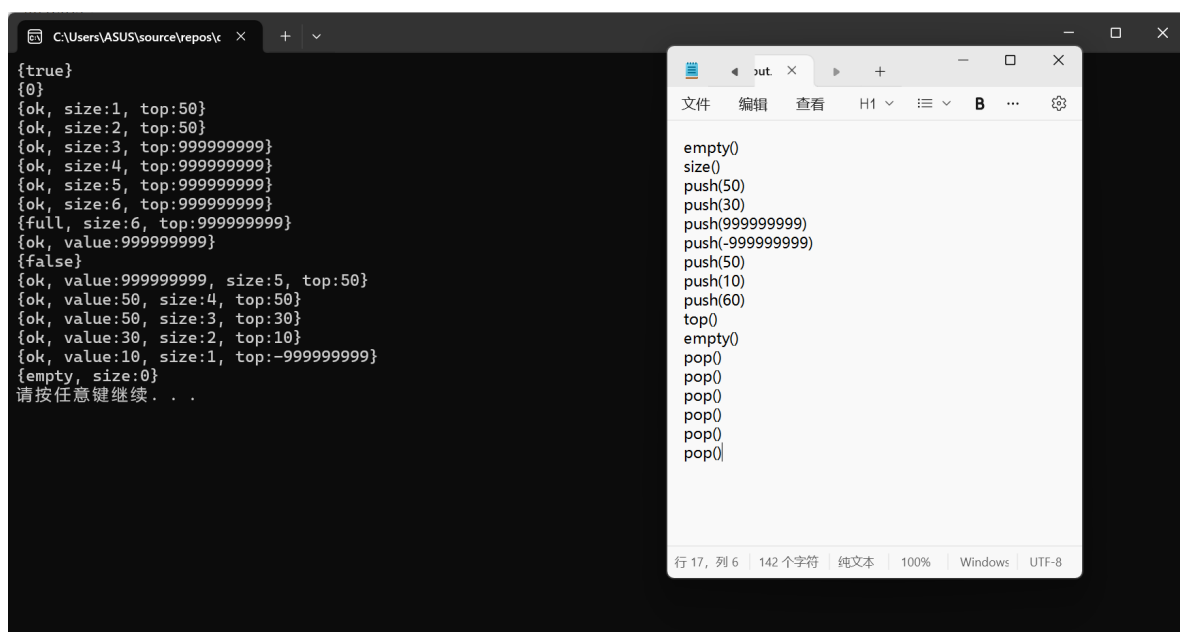
```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 #include <fstream>
5 #include <sstream>
6 using namespace std;
7 template<typename T>
8 class myHeap {
9 private:
10     vector<T> arr;
11     int cnt;
12     int maxSize;
13     T maxVal;
14     T minVal;
15     void up(int pos) {
16         while (pos > 1 && arr[pos] > arr[pos / 2]) {
17             swap(arr[pos], arr[pos / 2]);
18             pos /= 2;
19         }
20     }
21     void down(int pos) {
22         while (pos * 2 <= cnt) {
23             int kid = pos * 2;
24             if (kid < cnt && arr[kid] < arr[kid + 1]) {
25                 kid++;
26             }
27             if (arr[pos] >= arr[kid])
28                 break;
29             swap(arr[pos], arr[kid]);
30             pos = kid;
31         }
32     }
33 public:
34     myHeap(int n, T big, T small) : maxSize(n), maxVal(big), minVal(small), cnt(0) {
35         arr.resize(2 * n + 2);
36         arr[0] = maxVal;
37         for (int i = n + 1; i <= 2 * n + 1; i++) {
38             arr[i] = minVal;
39         }
40     }
```

```
41     bool empty() {
42         return cnt == 0;
43     }
44     bool full() {
45         return cnt == maxSize;
46     }
47     int size() {
48         return cnt;
49     }
50     T top() {
51         if (empty()) throw "empty";
52         return arr[1];
53     }
54     void push(T x) {
55         if (full()) throw "full";
56         cnt++;
57         arr[cnt] = x;
58         up(cnt);
59     }
60     void pop() {
61         if (empty()) throw "empty";
62         arr[1] = arr[cnt];
63         cnt--;
64         if (cnt > 0) down(1);
65     }
66 };
67 int main() {
68     int cap = 6;
69     int maxElement = 1000000000;
70     int minElement = -1000000000;
71     myHeap<int> h(cap, maxElement, minElement);
72     ifstream inputFile("input.txt");
73     if (!inputFile.is_open()) {
74         cout << "WRONG" << endl;
75         system("pause");
76         return 1;
77     }
78     string line;
79     while (getline(inputFile, line)) {
80         try {
81             line.erase(remove(line.begin(), line.end(), ' '), line.end());
82             if (line.find("push(") == 0 && line.back() == ')') {
```

```
83     int x = stoi(line.substr(5, line.length() - 6));
84     if (h.full()) {
85         cout << "{full, size:" << h.size();
86         if (!h.empty()) {
87             cout << ", top:" << h.top();
88         }
89         cout << "}" << endl;
90     }
91     else {
92         h.push(x);
93         cout << "{ok, size:" << h.size();
94         if (!h.empty()) {
95             cout << ", top:" << h.top();
96         }
97         cout << "}" << endl;
98     }
99 }
100 else if (line == "pop()") {
101     if (h.empty()) {
102         cout << "{empty, size:0}" << endl;
103     }
104     else {
105         int top_val = h.top();
106         h.pop();
107         if (h.empty()) {
108             cout << "{empty, size:0}" << endl;
109         }
110         else {
111             cout << "{ok, value:" << top_val << ", size:" << h.size();
112             if (!h.empty()) {
113                 cout << ", top:" << h.top();
114             }
115             cout << "}" << endl;
116         }
117     }
118 }
119 else if (line == "top()") {
120     if (h.empty()) {
121         cout << "{empty}" << endl;
122     }
123     else {
124         cout << "{ok, value:" << h.top() << "}" << endl;
```

```
125     }
126 }
127 else if (line == "empty()") {
128     cout << "{" << (h.empty() ? "true" : "false") << "}" << endl;
129 }
130 else if (line == "size()") {
131     cout << "{" << h.size() << "}" << endl;
132 }
133 else {
134     cout << "WRONG " << line << endl;
135 }
136 }
137 catch (const char* msg) {
138     cout << "WRONG" << msg << endl;
139 }
140 catch (const exception& e) {
141     cout << "WRONG" << line << endl;
142 }
143 }
144 inputFile.close();
145 system("pause");
146 return 0;
147 }
```

1.5 测试用例截图



```
C:\Users\ASUS\source\repos\c x + v
{true}
{0}
{ok, size:1, top:50}
{ok, size:2, top:50}
{ok, size:3, top:999999999}
{ok, size:4, top:999999999}
{ok, size:5, top:999999999}
{ok, size:6, top:999999999}
{full, size:6, top:999999999}
{ok, value:999999999}
{false}
{ok, value:999999999, size:5, top:50}
{ok, value:50, size:4, top:50}
{ok, value:50, size:3, top:30}
{ok, value:30, size:2, top:10}
{ok, value:10, size:1, top:-999999999}
{empty, size:0}
请按任意键继续. . .

out x + v
empty()
size()
push(50)
push(30)
push(999999999)
push(-999999999)
push(50)
push(10)
push(60)
top()
empty()
pop()
pop()
pop()
pop()
pop()
pop()
```


1.6 与本节实现的比较

1.6.1 本节书中的最大堆实现方式

push 操作特点:

1. 需要动态扩容检查
2. 使用”上浮”操作, 通过 while 循环找到合适位置
3. 需要比较 $\text{currentNode} \neq 1$ 来检查是否到达根节点

pop 操作特点:

1. 需要显式调用析构函数 $\text{heap}[1].\sim T()$
2. 使用”下沉”操作, 通过 while 循环重建堆
3. 需要检查 $\text{child} < \text{heapSize}$ 来处理边界情况

1.6.2 本题的最大堆实现方式

本题的实现方式简化了书中实现方式。

1. 消除了显式的根节点检查: 由于 $\text{heap}[0] = \text{maxElement}$, 在上浮操作中, 当 $\text{pos}=1$ 时, $\text{arr}[1] > \text{arr}[0]$ 永远为假, 循环自动终止。
2. 简化了叶子节点检查: 由于 $\text{heap}[n+1:2n+1] = \text{minElement}$, 在下沉操作中, 当下沉到叶子节点时, 比较会自动停止。

不过同时, 本题的实现方式需要额外的内存空间, 数组大小从 $n+1$ 变成了 $2n+2$ 。

2 题目二

2.1 题目表述

设计一个完整的基于霍夫曼编码的压缩-解压缩软件包。测试你的代码。

2.2 代码 2 的测试用例

1. `compress("mixed.txt", "mixed.huf")`
→ ok, `in_bytes`: 原文件字节数 $M1$, `out_bytes`: $M2$
// 期望: $M2 < M1$ (出现大量重复字符, 应有明显压缩效果)
`decompress("mixed.huf", "mixed.out")`
→ ok, `out_bytes`: $M1$
比较文件内容:
`compare("mixed.txt", "mixed.out") -> equal:true`
// 期望: 原文与解压结果完全一致

2. `compress("one_char.txt", "one_char.huf")`
→ ok, in_bytes:6, out_bytes:K // K 一般会略小于 6, 但实现合规即可
`decompress("one_char.huf", "one_char.out")`
→ ok, out_bytes:6
`compare("one_char.txt", "one_char.out") -> equal:true`
3. `compress("empty.txt", "empty.huf")`
→ ok, in_bytes:0, out_bytes:0
`decompress("empty.huf", "empty.out")`
→ ok, out_bytes:0
`compare("empty.txt", "empty.out") -> equal:true`

2.3 思路

2.3.1 哈夫曼树构建

1. 遍历输入数据, 统计每个字符出现的频率
2. 为每个字符创建叶子节点, 按频率排序
3. 处理特殊情况
 - 空文件直接返回空指针
 - 单一字符创建虚拟根节点, 确保编码正确
4. 构建哈夫曼树

2.3.2 生成编码表

递归生成编码, 左路径添加 0, 右路径添加 1

2.3.3 压缩处理

1. 将原始数据转换为哈夫曼编码的二进制串
2. 添加填充位, 处理字节对齐
3. 将二进制串分组成字节写入输出

2.3.4 解压缩处理

1. 基于压缩后的文件, 从序列化数据中递归重建哈夫曼树
2. 将压缩数据转换回二进制串

2.4 代码

```
1  #include <iostream>
2  #include <fstream>
3  #include <vector>
4  #include <queue>
5  #include <map>
6  #include <bitset>
7  #include <memory>
8  #include <string>
9  #include <iomanip>
10 using namespace std;
11 struct Node {
12     unsigned char ch;
13     int freq;
14     shared_ptr<Node> left, right;
15     Node(unsigned char c, int f)
16         : ch(c), freq(f), left(nullptr), right(nullptr) {}
17 };
18
19 struct Cmp {
20     bool operator()(const shared_ptr<Node>& a,
21                     const shared_ptr<Node>& b) const {
22         return a->freq > b->freq;
23     }
24 };
25
26 shared_ptr<Node> buildTree(const vector<unsigned char>& data) {
27     map<unsigned char, int> fr;
28     for (unsigned char c : data) fr[c]++;
29     priority_queue<shared_ptr<Node>, vector<shared_ptr<Node>>, Cmp> pq;
30     for (auto& p : fr) pq.push(make_shared<Node>(p.first, p.second));
31     if (pq.empty())
32         return nullptr;
33     if (pq.size() == 1) {
34         auto lone = pq.top(); pq.pop();
35         auto rt = make_shared<Node>('\0', lone->freq);
36         rt->left = lone;
37         pq.push(rt);
38     }
39     while (pq.size() > 1) {
40         auto l = pq.top(); pq.pop();
```

```
41     auto r = pq.top(); pq.pop();
42     auto m = make_shared<Node>('\0', l->freq + r->freq);
43     m->left = l; m->right = r;
44     pq.push(m);
45 }
46 return pq.top();
47 }
48
49 map<unsigned char, string> codeMap;
50 void genCodes(shared_ptr<Node> n, string s) {
51     if (!n) return;
52     if (!n->left && !n->right) {
53         codeMap[n->ch] = s.empty() ? "0" : s;
54         return;
55     }
56     genCodes(n->left, s + "0");
57     genCodes(n->right, s + "1");
58 }
59
60 void serializeTree(shared_ptr<Node> node, vector<unsigned char>& output) {
61     if (!node) return;
62     if (!node->left && !node->right) {
63         output.push_back(1);
64         output.push_back(node->ch);
65     }
66     else {
67         output.push_back(0);
68         serializeTree(node->left, output);
69         serializeTree(node->right, output);
70     }
71 }
72 shared_ptr<Node> deserializeTree(const vector<unsigned char>& data, size_t& index) {
73     if (index >= data.size()) return nullptr;
74     unsigned char flag = data[index++];
75     if (flag == 1) {
76         if (index >= data.size()) return nullptr;
77         unsigned char ch = data[index++];
78         return make_shared<Node>(ch, 0);
79     }
80     else {
81         auto node = make_shared<Node>('\0', 0);
82         node->left = deserializeTree(data, index);
```

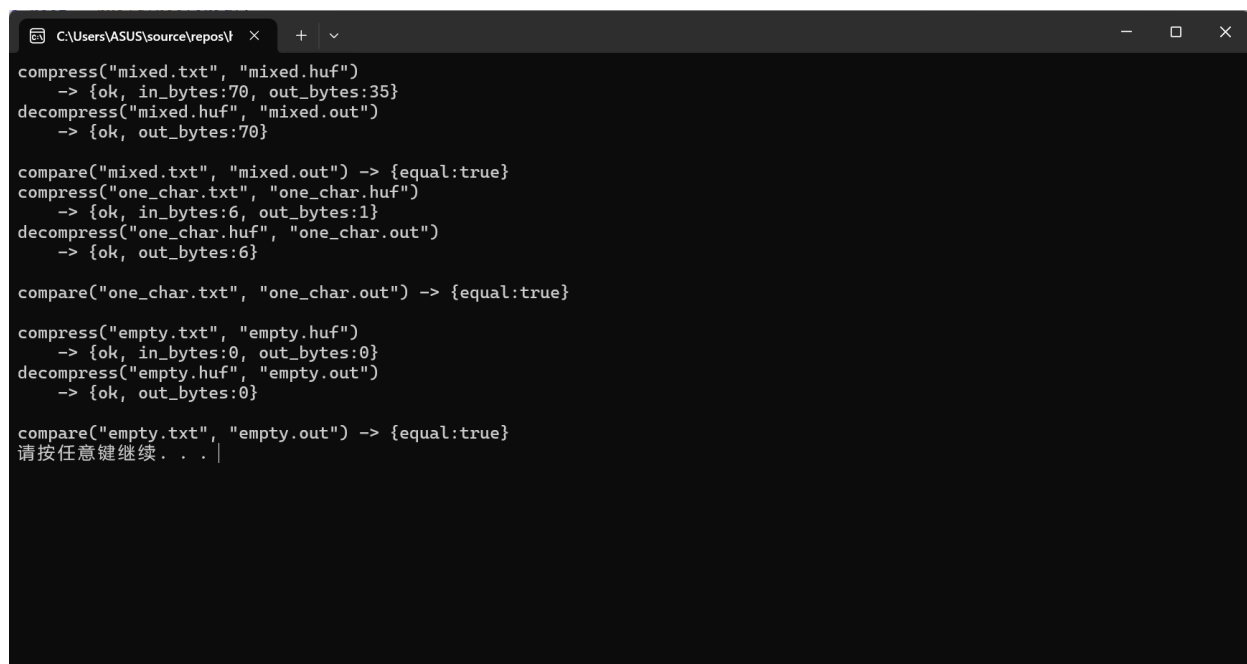
```
83     node->right = deserializeTree(data, index);
84     return node;
85 }
86 }
87
88 vector<unsigned char> compressRaw(const vector<unsigned char>& in,
89     int& padBits) {
90     codeMap.clear();
91     auto root = buildTree(in);
92     genCodes(root, "");
93     string bits;
94     for (unsigned char c : in) bits += codeMap[c];
95     padBits = (8 - (bits.size() % 8)) % 8;
96     bits += string(padBits, '0');
97     vector<unsigned char> out;
98     for (size_t i = 0; i < bits.size(); i += 8) {
99         string byte = bits.substr(i, 8);
100         out.push_back(static_cast<unsigned char>(bitset<8>(byte).to_ulong()));
101     }
102     return out;
103 }
104
105 vector<unsigned char> decompressRaw(const vector<unsigned char>& comp,
106     int padBits,
107     shared_ptr<Node> root) {
108     if (!root || comp.empty())
109         return {};
110     string bits;
111     for (unsigned char b : comp)
112         bits += bitset<8>(b).to_string();
113     if (padBits)
114         bits.erase(bits.end() - padBits, bits.end());
115     vector<unsigned char> out;
116     auto cur = root;
117     for (char c : bits) {
118         cur = (c == '0') ? cur->left : cur->right;
119         if (!cur->left && !cur->right) {
120             out.push_back(cur->ch);
121             cur = root;
122         }
123     }
124     return out;
```

```
125 }
126
127 vector<unsigned char> readFile(const string& name) {
128     ifstream f(name, ios::binary | ios::ate);
129     if (!f)
130         throw runtime_error("cannot open " + name);
131     size_t n = f.tellg();
132     f.seekg(0);
133     vector<unsigned char> data(n);
134     f.read(reinterpret_cast<char*>(data.data()), n);
135     return data;
136 }
137
138 void writeFile(const string& name, const vector<unsigned char>& data) {
139     ofstream f(name, ios::binary);
140     f.write(reinterpret_cast<const char*>(data.data()), data.size());
141 }
142
143 bool compareFiles(const string& name1, const string& name2) {
144     auto data1 = readFile(name1);
145     auto data2 = readFile(name2);
146     return data1 == data2;
147 }
148 void compress(const string& inputFile, const string& outputFile) {
149     auto orig = readFile(inputFile);
150     size_t origSize = orig.size();
151     int pad = 0;
152     auto root = buildTree(orig);
153     auto compressed = compressRaw(orig, pad);
154     size_t compSize = compressed.size();
155     vector<unsigned char> treeData;
156     serializeTree(root, treeData);
157     ofstream f(outputFile, ios::binary);
158     f.put(static_cast<unsigned char>(pad));
159     uint16_t treeSize = treeData.size();
160     f.put(static_cast<unsigned char>(treeSize & 0xFF));
161     f.put(static_cast<unsigned char>((treeSize >> 8) & 0xFF));
162     f.write(reinterpret_cast<const char*>(treeData.data()), treeData.size());
163     f.write(reinterpret_cast<const char*>(compressed.data()), compressed.size());
164     size_t reportedOutputSize = compSize; // 包含压缩数据
165     cout << "compress(\"" << inputFile << "\", \"" << outputFile << "\")" << endl;
166     cout << "  -> {ok, in_bytes:" << origSize << ", out_bytes:" <<
```

```
        reportedOutputSize << "}" << endl;
167 }
168
169 void decompress(const string& inputFile, const string& outputFile) {
170     auto data = readFile(inputFile);
171     if (data.empty()) {
172         writeFile(outputFile, {});
173         cout << "decompress(\"" << inputFile << "\", \"" << outputFile << "\")" <<
            endl;
174         cout << "  -> {ok, out_bytes:0}" << endl << endl;
175         return;
176     }
177     size_t index = 0;
178     int padBits = data[index++];
179     uint16_t treeSize = data[index++];
180     treeSize |= (static_cast<uint16_t>(data[index++]) << 8);
181     if (index + treeSize > data.size()) {
182         throw runtime_error("WRONG");
183     }
184     vector<unsigned char> treeData(data.begin() + index, data.begin() + index +
        treeSize);
185     index += treeSize;
186     size_t treeIndex = 0;
187     auto root = deserializeTree(treeData, treeIndex);
188     vector<unsigned char> compData(data.begin() + index, data.end());
189     auto decompressed = decompressRaw(compData, padBits, root);
190     writeFile(outputFile, decompressed);
191     cout << "decompress(\"" << inputFile << "\", \"" << outputFile << "\")" << endl;
192     cout << "  -> {ok, out_bytes:" << decompressed.size() << "}" << endl << endl;
193 }
194
195 int main() {
196     try {
197         // 测试 mixed.txt
198         compress("mixed.txt", "mixed.huf");
199         decompress("mixed.huf", "mixed.out");
200         cout << "compare(\"mixed.txt\", \"mixed.out\") -> {equal:"
201             << (compareFiles("mixed.txt", "mixed.out") ? "true" : "false") << "}" <<
                endl;
202         // 测试 one_char.txt
203         compress("one_char.txt", "one_char.huf");
204         decompress("one_char.huf", "one_char.out");
```

```
205     cout << "compare(\"one_char.txt\", \"one_char.out\") -> {equal:"
206         << (compareFiles("one_char.txt", "one_char.out") ? "true" : "false") <<
            "}" << endl << endl;
207     // 测试 empty.txt
208     compress("empty.txt", "empty.huf");
209     decompress("empty.huf", "empty.out");
210     cout << "compare(\"empty.txt\", \"empty.out\") -> {equal:"
211         << (compareFiles("empty.txt", "empty.out") ? "true" : "false") << "}" <<
            endl;
212 }
213 catch (const exception& e) {
214     cerr << "Error: " << e.what() << "\n";
215 }
216 system("pause");
217 return 0;
218 }
```

2.5 测试用例截图



```
C:\Users\ASUS\source\repos\ < + >
compress("mixed.txt", "mixed.huf")
-> {ok, in_bytes:70, out_bytes:35}
decompress("mixed.huf", "mixed.out")
-> {ok, out_bytes:70}

compare("mixed.txt", "mixed.out") -> {equal:true}
compress("one_char.txt", "one_char.huf")
-> {ok, in_bytes:6, out_bytes:1}
decompress("one_char.huf", "one_char.out")
-> {ok, out_bytes:6}

compare("one_char.txt", "one_char.out") -> {equal:true}

compress("empty.txt", "empty.huf")
-> {ok, in_bytes:0, out_bytes:0}
decompress("empty.huf", "empty.out")
-> {ok, out_bytes:0}

compare("empty.txt", "empty.out") -> {equal:true}
请按任意键继续. . . |
```